



IEEE Standard for Information Technology—Systems Design— Software Design Descriptions

IEEE Computer Society

Sponsored by the
Software & Systems Engineering Standards Committee

1016

IEEE 3 Park Avenue New York, NY 10016-5997, USA 20 July 2009

IEEE Std 1016™-2009

IEEE Standard for Information Technology— Systems Design—Software Design Descriptions

IMPORTANT NOTICE: This standard is not intended to ensure safety, security, health, or environmental protection in all circumstances. Implementers of the standard are responsible for determining appropriate safety, security, environmental, and health practices or regulatory requirements.

This IEEE document is made available for use subject to important notices and legal disclaimers. These notices and disclaimers appear in all publications containing this document and may be found under the heading “Important Notice” or “Important Notices and Disclaimers Concerning IEEE Documents.” They can also be obtained on request from IEEE or viewed at <http://standards.ieee.org/IPR/disclaimers.html>.

Change History

Version	Date	Author	Description
1.0	June 2026	SmartPharma Dev Team	Initial release — complete SDD for Smart Pharma System v1.0

Table of Contents

Change History	2
Table of Contents	3
1. Overview	6
1.1 Scope	6
1.2 Purpose	6
1.3 Intended Audience	6
1.4 Conformance	7
1.5 References	7
2. Definitions	8
3. Conceptual Model for the Smart Pharma Design Description	9
3.1 Smart Pharma System in Context	9
3.2 Design Stakeholders and Their Concerns	11
3.3 Design Description Life Cycle Role	11
4. Design Description Information Content	12
4.1 SDD Identification	12
4.2 Functional Module Design	12
4.2.1 Authentication Module Design	12
4.2.2 Inventory Module Design	12
4.2.3 Attendance Module Design	12
4.2.4 Salary Module Design	12
4.2.5 Task Management Module Design	12
4.2.6 Reporting and Audit Module Design	12
4.3 Design Stakeholders and Their Concerns	13
4.4 Design Views	13
4.5 Design Viewpoints	13
4.6 Design Elements	13
4.6.1 Design Entities	13
4.6.2 Design Attributes	14
4.6.3 Design Relationships	14
4.6.4 Design Constraints	14
4.7 Design Overlays	14
4.8 Design Rationale	15
4.9 Design Languages	15
5. Design Viewpoints	16
5.1 Introduction to Design Viewpoints	16

5.2 Context Viewpoint	16
5.2.1 Viewpoint Definition	16
5.2.2 Design Concerns	16
5.2.3 Context Design View	16
5.2.4 Design Rationale	16
5.3 Composition Viewpoint	16
5.3.1 Viewpoint Definition	16
5.3.2 Design Concerns	16
5.3.3 Composition Design View — Service Level	17
5.4 Logical Viewpoint	17
5.4.1 Viewpoint Definition	17
5.4.2 Design Concerns	17
5.4.3 Core Service Classes	17
5.5 Dependency Viewpoint	18
5.5.1 Viewpoint Definition	18
5.5.2 Inter-Service Communication Dependencies	18
5.5.3 Critical Third-Party Library Dependencies	18
5.6 Information Viewpoint	18
5.6.1 Viewpoint Definition	18
5.6.2 Primary Database Schema — MongoDB	19
5.7 Interface Viewpoint	19
5.7.1 Viewpoint Definition	19
5.7.2 Base URL and Common Conventions	19
5.7.3 Endpoint Specifications	19
5.8 Interaction Viewpoint	20
5.8.1 Viewpoint Definition	20
5.8.2 Interaction Scenario 1: Employee Login and Dashboard Load	20
5.8.3 Interaction Scenario 2: Attendance Check-In	20
5.9 State Dynamics Viewpoint	20
5.9.1 Viewpoint Definition	20
5.9.2 Attendance State Machine	20
5.9.3 Task State Machine	20
5.10 Algorithm Viewpoint	21
5.10.1 Viewpoint Definition	21
5.10.2 Salary Computation Algorithm	21
5.10.3 QR Code Generation Algorithm	21

5.11 Resource Viewpoint.....	21
5.11.1 Viewpoint Definition	21
5.11.2 Compute Resource Design.....	21
5.11.3 Deployment Design.....	21
6. Security Design.....	23
6.1 Authentication and Authorization Architecture	23
6.2 Encryption Design	23
6.3 Input Validation Design.....	23
6.4 Multi-Tenancy Design	23
7. Design Overlays.....	24
7.1 Security Overlay	24
7.2 Observability Overlay	24
7.3 Multi-Tenancy Overlay.....	24
8. Design Rationale.....	25
8.1 MERN Stack Architecture Decision	25
8.2 MongoDB over Relational Database	25
8.3 MVC Design Pattern.....	25
8.4 Service Layer for Business Logic	25
9. Design Languages.....	26
10. Summary	27
10.1 Key Design Characteristics.....	27
10.2 Outstanding Design Items.....	27
Annex A.....	28
Bibliography	28
Annex B.....	29
Conforming Design Language Descriptions.....	29
Annex C	30
SDD Structure Map — Smart Pharma to IEEE 1016-2009.....	30

1. Overview

1.1 Scope

This Software Design Description (SDD) documents the design of the Smart Pharma System — a full-stack MERN-based web application developed to automate pharmaceutical packaging operations. The platform delivers centralized employee management, attendance tracking, salary computation, inventory management for foils and cylinders, QR/barcode generation, task assignment and tracking, audit logging, and analytics reporting.

This SDD applies to the entire Smart Pharma System, encompassing: the React.js frontend application, the Node.js/Express.js backend API, the MongoDB document store, and all integrations with JWT-based authentication and Winston logging. It is applicable to both the current production release (v1.0) and serves as the foundational reference for future enhancements.

This document does not prescribe specific development methodologies, configuration management procedures, or quality assurance workflows beyond what is described as part of the design itself. It is not a requirements specification, a test plan, or an operational manual, though it informs the preparation of all of these.

1.2 Purpose

The purpose of this SDD is to specify the requirements on the information content and organization of the Smart Pharma system design, in conformance with IEEE Std 1016-2009. Specifically, this document:

- Establishes the complete architectural blueprint of the Smart Pharma platform, including its three-tier MVC decomposition, REST API contracts, and integration points.
- Defines all major design viewpoints — Context, Composition, Logical, Dependency, Information, Interface, Interaction, State Dynamics, Algorithm, and Resource — each addressing a distinct set of design concerns raised by stakeholders.
- Provides a traceable mapping from system requirements and stakeholder concerns to design decisions, enabling verification and validation activities.
- Documents design rationale, capturing the reasoning behind key architectural choices, technology selections, and security mechanisms.
- Serves as the primary reference for all system implementers, integrators, testers, and maintenance engineers throughout the system lifecycle.

1.3 Intended Audience

This SDD is intended for the following design stakeholders:

Table 1: Stakeholder Roles and Primary Interests

Stakeholder Role	Primary Interest in this SDD
Software Architects	System decomposition, three-tier architecture, module boundaries, scalability strategy
Backend Engineers	Express.js service design, REST API contracts, MongoDB data models, business logic specification
Frontend Developers	React.js component architecture, state management, role-based navigation, UI flow
Database Engineers	MongoDB schema design, Mongoose ODM patterns, multi-tenancy isolation, audit trail

QA / Test Engineers	Interface contracts, data flows, algorithmic logic, state transitions for test design
Security Engineers	JWT authentication design, bcrypt hashing, RBAC middleware, input validation
Technical Project Managers	Scope of design, component responsibilities, external dependencies

1.4 Conformance

This SDD conforms to IEEE Std 1016-2009 if it satisfies all requirements in Clause 4 (Design Description Information Content) and Clause 5 (Design Viewpoints) of that standard. Requirements are denoted by the verb shall throughout this document.

1.5 References

The following references influenced the software design of the Smart Pharma System:

- IEEE Std 1016-2009 – Software Design Description
- IEEE Std 12207-2008 – Systems and Software Engineering — Software Life Cycle Processes
- IEEE Std 1012-2004 – Software Verification and Validation
- React.js Documentation – Frontend component framework
- Node.js Documentation – Backend JavaScript runtime
- Express.js Documentation – REST API framework
- MongoDB Documentation – NoSQL document store and query processing
- Mongoose Documentation – MongoDB ODM integration
- JWT (JSON Web Token) Documentation – Secure authentication and authorization
- bcrypt Documentation – Password hashing library
- Winston Logger Documentation – Application logging framework
- express-validator Documentation – Input validation middleware
- express-rate-limit Documentation – API rate limiting

2. Definitions

For the purposes of this project, the following terms and definitions apply. These definitions align with IEEE standards to ensure technical clarity for the Smart Pharma system design.

design attribute: A design attribute represents a characteristic or property of a software design element. Examples include performance, reliability, scalability, and security. These attributes help evaluate how well the software system meets requirements.

design concern: A design concern is an important issue or area of interest related to the software design. It may include aspects such as maintainability, usability, security, or performance that stakeholders care about during development.

design constraint: A design constraint is a rule, limitation, or restriction applied to the software design. Constraints may come from hardware limits, business rules, regulations, or technology requirements that the system must follow.

design element: A design element is any component included in a design view, such as entities, relationships, attributes, or constraints. These elements together describe the structure and behavior of the software system.

design entity: A design entity is a distinct part of the software design that performs a specific role or function. Examples include modules, classes, databases, or services that work together within the system.

design overlay: A design overlay provides additional or detailed information on top of an existing design view. It helps designers represent advanced details or modifications without changing the original design structure.

design rationale: Design rationale explains the reasoning behind design decisions made during software development. It includes alternatives considered, trade-offs analyzed, and justifications for choosing a particular solution.

design relationship: A design relationship describes the connection or interaction between two or more design entities. These relationships help show dependencies, communication, or data flow within the system design.

design stakeholder: A design stakeholder is any person, group, or organization interested in the software design. Stakeholders may include developers, managers, users, clients, or testers who are affected by the system.

design subject: The design subject refers to the software system, application, or component being designed and documented. It is the main focus of the Software Design Description (SDD).

designer: A designer is the individual or team responsible for creating and documenting the software design. Designers ensure that the system structure and functionality meet stakeholder requirements.

design view: A design view is a representation of the software design created to address specific design concerns. Different views may focus on architecture, functionality, data flow, or deployment aspects of the system.

design viewpoint: A design viewpoint defines the rules, methods, and conventions used to create a design view. It helps maintain consistency and ensures that the design information is clearly understood by stakeholders.

diagram (type): A diagram is a graphical representation of software design elements and their relationships. It visually explains parts of the system design, making the structure and workflow easier to understand.

3. Conceptual Model for the Smart Pharma Design Description

This clause establishes a conceptual model for the Smart Pharma SDD. The conceptual model includes basic terms and concepts of SDD, the context in which SDDs are prepared and used, the stakeholders who use them, and how they are used.

3.1 Smart Pharma System in Context

The Smart Pharma pharmaceutical packaging management platform is a conceptualization of a design subject that embodies its essential characteristics; demonstrates a means to fulfill its operational and system requirements; serves as a basis for analysis and evaluation; and can be used to guide its implementation. The design subject is a MERN-stack web application integrating employee management, attendance tracking, salary computation, inventory management, task assignment, QR/barcode generation, and audit logging.

NOTE 1 — The Smart Pharma system as a design subject encompasses the complete system including its frontend subsystem (React.js SPA), backend subsystem (Node.js/Express.js REST API), database subsystem (MongoDB 6.x with Mongoose ODM), authentication subsystem (JWT-based), multi-tenancy subsystem (company-field isolation), and reporting subsystem. Each of these may itself be treated as a design subject at a lower level of concern.

The design concerns of the Smart Pharma system are expressed by its design stakeholders and arise from both operational requirements and contextual constraints:

- Functionality — employee registration and RBAC, daily attendance check-in/out, automated salary calculation, inventory tracking for foils and cylinders, QR/barcode generation, task lifecycle management, and comprehensive audit logging
- Security — JWT authentication, role-based access control (RBAC), bcrypt password hashing, multi-tenant data isolation, input validation, and rate limiting
- Reliability — Winston logging, health check endpoints, MongoDB session transactions for critical operations, and fault-tolerant API error handling
- Performance — sub-500ms API response, optimized MongoDB queries, Mongoose indexing, and React state management
- Maintainability — modular folder structure, MVC design pattern, service layer encapsulation, and ESLint code quality enforcement

The key concepts of SDD are depicted in Figure 1a and Figure 1b.

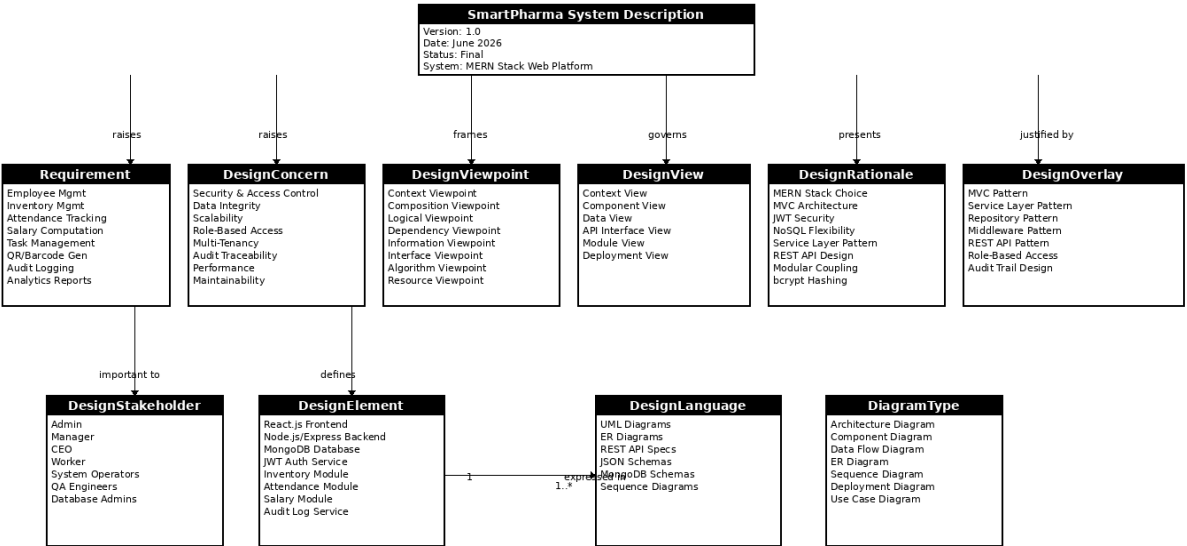


Figure 1a — Conceptual model: top view

Figure 1a—Conceptual model: top view

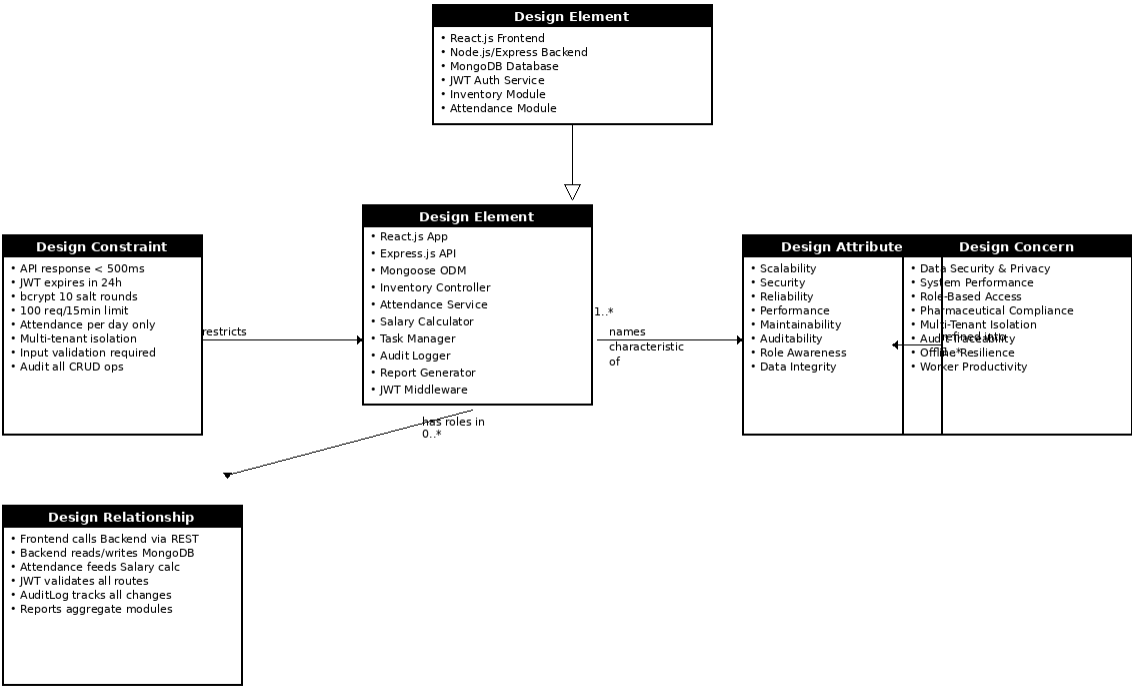


Figure 1b — Conceptual model: design elements

Figure 1b—Conceptual model: design elements

3.2 Design Stakeholders and Their Concerns

The Smart Pharma SDD identifies multiple stakeholders associated with the pharmaceutical packaging management platform. The primary design stakeholders include: Administrators, Managers, CEOs, Workers, System Operators, Database Administrators, and QA/Test Engineers.

Table 2: Stakeholder Design Concerns

Stakeholder	Design Concerns
Admin	User registration, role assignment, full system access, audit log visibility, user management, multi-tenant isolation
Manager	Task creation and assignment, inventory management, attendance monitoring, report generation, audit log access
CEO	Dashboard analytics, salary reports, attendance summaries, audit log visibility, business intelligence
Worker	Personal check-in/out, assigned task visibility, own profile management, daily task status updates
System Operators	System availability, Winston logging, health check endpoint, API performance monitoring
Database Administrators	MongoDB schema integrity, Mongoose validation, multi-tenancy company-field isolation, data backup
QA/Test Engineers	API contract testing via Postman, RBAC validation, attendance logic, salary computation accuracy, audit completeness

3.3 Design Description Life Cycle Role

The Smart Pharma SDD was prepared to support the active development of the pharmaceutical packaging platform. The SDD serves as a reference for implementation, testing, deployment, and future maintenance of the system.

4. Design Description Information Content

4.1 SDD Identification

Date of issue and status

Date: June 2026 Status: Production-Ready / Approved for Academic Submission

Scope

The SDD defines the architecture, components, interfaces, workflows, database structure, and deployment of the Smart Pharma MERN-stack web application for pharmaceutical packaging operations management.

Issuing organization

CSE23CL302 – CASE Tools Laboratory, Sri Ramachandra Faculty of Engineering and Technology, SRIHER, Porur, Chennai – 600116

Authorship

Prepared by Bharatraj (E0323054) and Yokesnandha (E0323041) under the supervision of Ms. Yamini K, Assistant Professor, Dept. of Cybersecurity & IoT.

4.2 Functional Module Design

The Smart Pharma System is decomposed into six specialized modules with high cohesion and loose coupling. Each module owns its routes, controllers, and Mongoose models.

4.2.1 Authentication Module Design

The Authentication module handles user registration by administrators, login with JWT issuance, password hashing with bcrypt (10+ salt rounds), and role-based access control (RBAC). Roles include admin, manager, ceo, and worker. JWT tokens expire in 24 hours and are validated via middleware on every protected route.

4.2.2 Inventory Module Design

The Inventory module manages foil stock (blister, strip) and cylinder stock. Each item carries a type, size, weight, QR code string, version counter for audit trail, and tenant identifier. The module generates QR/barcode strings on item creation and supports full CRUD operations with admin/manager authorization.

4.2.3 Attendance Module Design

The Attendance module records daily check-in and check-out timestamps for workers. Working hours are automatically computed on checkout. Records include status (Present/Absent/Half Day), hourly rate, and earnings (hours × rate). Only one attendance record per worker per day is permitted.

4.2.4 Salary Module Design

The Salary module reads attendance records via service call to compute total earnings per worker per period. Salary reports aggregate attendance data and apply configured hourly rates. Salary computation is decoupled from the Attendance module through a service interface.

4.2.5 Task Management Module Design

The Task module supports task creation with assignment to workers, tracking of task status through pending, in-progress, and completed states, and linkage to foil type specifications. Tasks record the assigning manager, required material quantity, and Mongoose timestamps for lifecycle tracking.

4.2.6 Reporting and Audit Module Design

The Reporting module aggregates data from Attendance, Inventory, and Salary modules via API calls. The Audit module persists all create, edit, delete, login, and logout events to the AuditLog collection with actor identity, role, entity type, item reference, and change summary.

4.3 Design Stakeholders and Their Concerns

Refer to Table 2 in Section 3.2 for the complete stakeholder design concerns mapping.

4.4 Design Views

The Smart Pharma SDD is organized into multiple design views that collectively describe the complete architecture and behavior of the platform. The major design views include: System Architecture View, Frontend Component View, Backend Microservice View, Database Design View, Authentication Workflow View, Task Lifecycle View, DevOps and Deployment View, and Security and RBAC View.

4.5 Design Viewpoints

Table 5: Design Viewpoints Summary

Design Viewpoint	Design Concerns	Example Design Language
Context (5.2)	System boundary, external actors, integration points, operational environment	UML Use Case Diagram
Composition (5.3)	Module decomposition, component responsibilities, three-tier separation	UML Component Diagram
Logical (5.4)	Class structure, MongoDB schema design, functional responsibilities	UML Class Diagram, Mongoose Schema
Dependency (5.5)	Module coupling, REST API dependencies, third-party libraries	Dependency graph, tabular notation
Information (5.6)	MongoDB collections, schema definitions, data flows, persistence strategy	ER Diagram, MongoDB schema notation
Interface (5.7)	REST API contracts, request/response schemas, endpoint specifications	REST API notation, JSON schema
Interaction (5.8)	Message flows, request lifecycle, service orchestration	UML Sequence Diagram
State Dynamics (5.9)	Attendance lifecycle, task state machine, session management	UML State Machine Diagram
Algorithm (5.10)	Salary computation, QR generation, attendance validation logic	Pseudocode, flowchart
Resource (5.11)	Memory management, rate limiting, containerization, deployment	Tabular notation, deployment description

4.6 Design Elements

The primary design elements identified in the Smart Pharma system include: React.js Frontend Application, Node.js/Express.js Backend Service, Inventory Management Module, Attendance Tracking Module, Salary Computation Module, Task Management Module, Reporting & Audit Module, Authentication Service, and MongoDB Data Layer.

4.6.1 Design Entities

Table 3: Smart Pharma Design Entities

Design Entity	Type	Purpose
React.js SPA	Frontend Application	Provides role-aware UI for all user types on desktop and tablet

Express.js Backend	Backend Service	Processes REST API requests, enforces RBAC, and executes business logic
Inventory Module	Business Module	Manages foil/cylinder stock, QR generation, and version audit trail
Attendance Module	Business Module	Records daily check-in/out and computes working hours and earnings
Salary Module	Business Module	Computes payroll from attendance data and configured hourly rates
Task Module	Business Module	Manages task lifecycle from creation through completion with worker assignment
Reporting Module	Analytics Service	Aggregates cross-module data for attendance, inventory, and salary reports
AuditLog Service	Compliance Service	Records all CRUD, login, and logout events for traceability
MongoDB Database	Database System	Stores all collections with Mongoose schemas and multi-tenant isolation
JWT Middleware	Security Service	Validates Bearer tokens and enforces role-based route access

4.6.2 Design Attributes

The major attributes associated with Smart Pharma design elements include: Scalability, Security, Availability, Latency, Response time, Role-awareness, Fault tolerance, Multi-tenancy, Auditability, and Data integrity.

4.6.3 Design Relationships

Examples of design relationships include:

- The React.js frontend communicates with the Express.js backend through REST APIs via Axios/Fetch.
- The Express.js backend interacts with MongoDB through Mongoose ODM for all data persistence.
- The Salary Module reads attendance data through a service call to the Attendance Module.
- The Reporting Module collects data from multiple modules via API calls.
- The JWT Middleware validates tokens before any protected route controller executes.
- The AuditLog Service is triggered by all create, edit, delete, login, and logout operations.

4.6.4 Design Constraints

Major design constraints include:

- API response time must remain below 500 milliseconds under normal load.
- JWT tokens expire in 24 hours; no refresh token mechanism in v1.0.
- Rate limiting: 100 requests per 15-minute window per IP.
- Only one attendance record per worker per day is permitted.
- All database queries must include company field filter for multi-tenancy isolation.
- Passwords must never be stored in plaintext; bcrypt with 10+ salt rounds required.
- All CRUD, login, and logout operations must generate an AuditLog entry.

4.7 Design Overlays

The major design overlays include:

- Security Overlay — Illustrates JWT authentication flow, RBAC middleware chain, bcrypt hashing, and rate limiting.
- Audit Trail Overlay — Represents Winston logging, AuditLog collection population, and event traceability.
- Multi-Tenancy Overlay — Shows company-field isolation across all MongoDB collections and query patterns.
- Reporting Overlay — Displays cross-module data aggregation for attendance, salary, and inventory analytics.

4.8 Design Rationale

- MERN Stack was selected to enable full-stack JavaScript development, reducing context switching and improving team productivity.
- MongoDB was selected for its schema flexibility, document-oriented storage well-suited to hierarchical pharmaceutical data.
- MVC pattern was adopted to separate UI rendering, business logic, and data access for maintainability.
- Service Layer pattern encapsulates business logic outside controllers to improve testability and reuse.
- JWT was selected for stateless authentication, supporting horizontal scaling without server-side session storage.
- Multi-tenancy via company field was selected for simplicity and deployment efficiency for a single-instance multi-company solution.

4.9 Design Languages

The Smart Pharma SDD uses: Unified Modeling Language (UML) for use case, class, and sequence diagrams; Entity-Relationship Diagrams (ERD) for MongoDB collection design; REST API Specifications for endpoint documentation; JSON Schema Definitions for request/response structures; MongoDB Mongoose Schema Notation for collection definitions; and Flowcharts for algorithm representation.

5. Design Viewpoints

5.1 Introduction to Design Viewpoints

This section specifies the design viewpoints selected for the Smart Pharma SDD. Each viewpoint focuses on a distinct set of design concerns and introduces a set of descriptive resources used to construct and interpret the corresponding design view. For each design view, there is exactly one governing design viewpoint.

5.2 Context Viewpoint

The Context viewpoint describes the Smart Pharma System as a black-box web application interacting with external actors such as admins, managers, CEOs, and workers. It identifies the system boundary, offered services, and information exchange between the platform and its operational environment.

5.2.1 Viewpoint Definition

The Context viewpoint provides a black-box perspective on the Smart Pharma System, abstracting all internal implementation details while focusing on interactions between external actors and the platform. The system boundary separates the Smart Pharma web application from browsers, MongoDB Atlas, and the Node.js hosting environment.

5.2.2 Design Concerns

The primary design concerns addressed by the Context viewpoint include: system boundary definition, identification of all external actors (Admin, Manager, CEO, Worker), services offered (authentication, inventory, attendance, salary, tasks, reports, audit), and information flows (login requests, inventory CRUD, attendance records, salary reports, task assignments, audit events).

5.2.3 Context Design View

Actors: Admin, Manager, CEO, Worker, System Operator.

Services: User authentication and authorization, inventory management, attendance recording, salary computation, task lifecycle management, QR/barcode generation, analytics reporting, audit logging.

Information Flows: User credentials → JWT tokens; inventory data → MongoDB; attendance timestamps → hours/earnings; task assignments → worker dashboards; audit events → AuditLog collection.

5.2.4 Design Rationale

The Context viewpoint was selected because the Smart Pharma System serves multiple distinct user roles with different access scopes. Defining the system boundary explicitly helps enforce multi-tenancy isolation and RBAC from the outermost layer of the design.

Fig 5.2 Context Viewpoint

5.3 Composition Viewpoint

The Composition viewpoint explains how the Smart Pharma System is structurally divided into major subsystems: the React.js frontend SPA, Node.js/Express.js backend API, MongoDB database layer, JWT authentication service, and Winston logging infrastructure.

5.3.1 Viewpoint Definition

The Composition viewpoint describes the three-tier decomposition of the Smart Pharma System. The platform follows a client-server MVC architecture where responsibilities are separated into independently manageable layers and modules.

5.3.2 Design Concerns

Key concerns include: allocation of functionality into reusable modules, separation of presentation, business logic, and data layers, modularization for scalability and maintainability, and independent deployment of frontend and backend.

5.3.3 Composition Design View — Service Level

Table 6: Component Functions

Component	Function
React.js Frontend	Role-aware UI rendering for all user types; state management via hooks
Express.js Backend	REST API gateway; JWT validation; RBAC enforcement; business logic execution
Authentication Service	User registration (admin), login, JWT issuance, bcrypt password hashing
Inventory Module	Foil/cylinder CRUD, QR code generation, version tracking, stock search
Attendance Module	Daily check-in/out recording, hours calculation, earnings computation
Salary Module	Payroll computation from attendance records; salary report generation
Task Module	Task creation/assignment, status tracking, material specification linking
Report Module	Cross-module data aggregation for attendance, inventory, salary analytics
Audit Service	System-wide event logging for all CRUD, login, and logout operations

Fig 5.3 Composition Viewpoint

5.4 Logical Viewpoint

The Logical viewpoint defines the object-oriented and schema structure of the Smart Pharma System using MongoDB collections, Mongoose schemas, Express controllers, and React components with their structural static relationships.

5.4.1 Viewpoint Definition

The Logical viewpoint focuses on data structures, class responsibilities, and logical relationships. It defines Mongoose schemas, React component hierarchies, Express controller responsibilities, and service module interfaces.

5.4.2 Design Concerns

Primary concerns include: reusable domain abstractions for pharmaceutical operations, user and employee data modeling, attendance and payroll data structures, inventory schema design, and relationship modeling between workers, tasks, and inventory items.

5.4.3 Core Service Classes

The major logical entities are: User (with role field), InventoryItem (foil/cylinder), AttendanceRecord, SalaryRecord, Task, AuditLogEntry, and Report. Key relationships include: User one-to-many

AttendanceRecord, User many-to-many Task (via assignedTo/assignedBy), InventoryItem referenced by Task.

Fig 5.4 Logical Viewpoint

5.5 Dependency Viewpoint

The Dependency viewpoint specifies the interconnections among Smart Pharma modules and their external libraries. It identifies how modules such as the Salary service, Attendance module, JWT middleware, and MongoDB depend on each other for correct operation.

5.5.1 Viewpoint Definition

The Dependency viewpoint identifies interconnections and operational dependencies. Key dependencies include: Salary Module on Attendance Module (via service call), all modules on JWT Middleware, all controllers on Mongoose Models, and Frontend on Backend REST API.

5.5.2 Inter-Service Communication Dependencies

Critical dependencies:

- React.js frontend depends on Express.js REST API endpoints.
- All backend routes depend on the JWT authenticator middleware and Role Guard.
- Salary Module depends on Attendance records via SalaryService.getAttendance().
- Report Module depends on Attendance, Inventory, and Salary modules via API calls.
- AuditLog Service is called by all controllers upon any data-modifying operation.

5.5.3 Critical Third-Party Library Dependencies

Table 7: Third-Party Dependencies

Library	Version	Purpose
express	4.x	HTTP server and REST API routing framework
mongoose	7.x	MongoDB ODM with schema validation and query building
jsonwebtoken	latest	JWT token creation, signing, and verification
bcryptjs	latest	Password hashing with configurable salt rounds
express-validator	latest	Request body sanitization and field validation
express-rate-limit	latest	IP-based API rate limiting (100 req/15 min)
winston	latest	Structured application logging with rotating log files
qrcode	latest	QR code string generation for inventory items

Fig 5.5 Dependency Viewpoint

5.6 Information Viewpoint

The Information viewpoint focuses on the management, storage, and organization of persistent pharmaceutical data within the Smart Pharma platform. The system stores user profiles, inventory records, attendance logs, salary records, task data, and audit trails using MongoDB 6.x.

5.6.1 Viewpoint Definition

The Information viewpoint describes the MongoDB collection schemas, document structures, field constraints, and relationships between data entities within the Smart Pharma System.

5.6.2 Primary Database Schema — MongoDB

Table 8: MongoDB Collection Schemas

Collection	Key Fields	Multi-Tenant
Users	_id, name, email, password (hashed), role (enum), company, phone, address, joiningDate	Yes (company)
Inventory	_id, type (enum), size, weight, company, qrCode, version, serial, timestamps	Yes (company)
Attendance	_id, worker (ref: User), company, date, status (enum), checkIn, checkOut, hours, rate, earnings	Yes (company)
Tasks	_id, title, company, worker (ref: User), assignedBy (ref: User), status (enum), foilType, size, weight, requiredKg, timestamps	Yes (company)
AuditLogs	_id, action (enum), type, item (ref), changedBy (ref: User), role, company, details, createdAt	Yes (company)

Fig 5.6 Information Viewpoint

5.7 Interface Viewpoint

The Interface viewpoint defines how Smart Pharma services and modules interact through REST APIs and user-facing interfaces. It provides interface specifications required by developers and testers for smooth communication between frontend and backend.

5.7.1 Viewpoint Definition

All interfaces follow REST conventions with JSON request/response bodies, Authorization: Bearer <token> headers for protected routes, and standard HTTP status codes (200, 201, 400, 401, 403, 404, 500).

5.7.2 Base URL and Common Conventions

Base URL: /api. All timestamps in ISO 8601 UTC format. All API responses return Content-Type: application/json.

5.7.3 Endpoint Specifications

Table 9: Authentication API

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create a new user account with role	Admin only
POST	/api/auth/login	Authenticate user and return JWT token	No
GET	/api/auth/me	Get current user profile	Yes
PUT	/api/auth/profile	Update user profile information	Yes
GET	/api/auth/users	List all users in the company	Admin/Manager

Table 10: Inventory API

Method	Endpoint	Description	Auth Required
GET	/api/inventory	List all inventory (foil/cylinder) items	Yes
POST	/api/inventory	Add new inventory item; auto-generates QR	Admin/Manager

PUT	/api/inventory/:id	Update existing inventory item	Admin/Manager
DELETE	/api/inventory/:id	Remove an inventory item	Admin only
GET	/api/inventory/:id/qr	Get QR code data for an item	Yes

Fig 5.7 Interface Viewpoint

5.8 Interaction Viewpoint

The Interaction viewpoint defines how Smart Pharma entities communicate during operational workflows such as user login, inventory management, attendance recording, and task completion. It describes REST request/response flows and middleware chain execution sequences.

5.8.1 Viewpoint Definition

The Interaction viewpoint documents message flows, middleware execution sequences, and data transformation steps for key system workflows.

5.8.2 Interaction Scenario 1: Employee Login and Dashboard Load

1. Worker submits email and password via Login Screen. 2. Frontend sends POST /api/auth/login. 3. Rate Limiter checks request count. 4. Express-validator sanitizes input. 5. AuthController queries MongoDB for user by email. 6. bcrypt.compare() validates password hash. 7. JWT token is signed and returned. 8. Frontend stores token; fetches dashboard data with Authorization header. 9. JWT Middleware verifies token on each subsequent request. 10. Role Guard validates worker role against required permissions.

5.8.3 Interaction Scenario 2: Attendance Check-In

1. Worker clicks Check-In on dashboard. 2. Frontend sends POST /api/attendance/checkin with Bearer token. 3. JWT Middleware validates token. 4. AttendanceController checks no existing record for today. 5. New Attendance document created with checkIn timestamp. 6. AuditLog entry created: action=create, type=attendance. 7. Response 201 returned. 8. Frontend displays success notification.

Fig 5.8 Interaction Viewpoint

5.9 State Dynamics Viewpoint

The State Dynamics viewpoint describes the dynamic behavior of Smart Pharma entities based on states, transitions, and events. It is especially relevant for the Attendance lifecycle and Task lifecycle workflows.

5.9.1 Viewpoint Definition

The State Dynamics viewpoint documents state machines for the key operational workflows in the Smart Pharma System.

5.9.2 Attendance State Machine

States: No Record → Checked-In → Checked-Out (terminal for the day). Transitions: checkIn event creates record in Checked-In state. checkOut event computes hours and earnings, transitions to Checked-Out. Once Checked-Out, no further transitions allowed for the day. Absent and Half Day statuses are set administratively.

5.9.3 Task State Machine

States: Draft → Pending → In-Progress → Completed. Transitions: task creation moves to Pending. Worker starts task: Pending → In-Progress. Worker marks complete: In-Progress → Completed. Admin/Manager may reset or delete in any state.

Fig 5.9 State Dynamics Viewpoint

5.10 Algorithm Viewpoint

The Algorithm viewpoint describes the internal processing logic and computational procedures used in Smart Pharma. It explains the sequence of operations for salary computation, QR generation, attendance hours calculation, and report aggregation.

5.10.1 Viewpoint Definition

The Algorithm viewpoint provides implementation-level detail for the key computational processes within the Smart Pharma platform.

5.10.2 Salary Computation Algorithm

Input: workerId, startDate, endDate. Process: 1. Query Attendance collection WHERE worker=workerId AND date BETWEEN startDate AND endDate AND status=Present. 2. Sum hours for each record. 3. Multiply total hours by configured hourly rate. 4. Apply Half Day factor (0.5) for records with status=Half Day. 5. Return { totalHours, totalEarnings, breakdown[] }. Constraint: attendance records must have checkOut timestamp; incomplete records excluded.

5.10.3 QR Code Generation Algorithm

Input: inventoryItem (type, size, weight, serial, company). Process: 1. Construct pipe-delimited string: type|size|weight|serial|company|timestamp. 2. Encode via qrcode library to generate QR string. 3. Store qrCode field in MongoDB document. 4. Increment version counter. 5. Create AuditLog entry for creation. Output: qrCode string, version number.

Fig 5.10 Algorithm Viewpoint

5.11 Resource Viewpoint

The Resource viewpoint models the utilization and performance characteristics of hardware, software, and network resources used by Smart Pharma. It focuses on deployment environment requirements, API rate limiting, and MongoDB resource management.

5.11.1 Viewpoint Definition

The Resource viewpoint documents the deployment environment requirements, hardware specifications, and resource constraints for the Smart Pharma System.

5.11.2 Compute Resource Design

Table 11: Environment Requirements

Component	Minimum	Recommended
Processor	Intel Core i3	Intel Core i5 / i7
RAM	8 GB	16 GB
Storage	256 GB HDD	512 GB SSD
Internet	10 Mbps	50 Mbps
OS	Windows 10/11 or Ubuntu 20.04+	Ubuntu 22.04 LTS (server)
Node.js	18.x LTS	20.x LTS
MongoDB	6.x	7.x

5.11.3 Deployment Design

Deployment follows a Node.js server with PM2 process manager for backend, Nginx static hosting for React frontend build, and MongoDB local or Atlas cloud instance. Winston logging writes to rotating daily

log files. Health check endpoint GET /api/health returns server status, DB connectivity, uptime, and memory usage.

Fig 5.11 Resource Viewpoint

6. Security Design

6.1 Authentication and Authorization Architecture

The Smart Pharma security architecture is built on JWT-based stateless authentication with role-based access control enforced at the middleware layer. Every protected API route passes through the following middleware pipeline: Rate Limiter → JSON Body Parser → CORS Policy → JWT Authenticator → Role Guard → Input Validator → Controller.

Table 12: Security Controls

Security Control	Implementation
Password Hashing	bcrypt with 10+ salt rounds — passwords never stored in plaintext
Token Authentication	JWT signed with secret key, expires in 24 hours
Role-Based Access Control	Middleware validates role on every protected route; roles: admin, manager, ceo, worker
Multi-Tenancy Isolation	Every DB query filtered by company field — users cannot access other tenants' data
Input Validation	express-validator sanitizes and validates all incoming API request bodies
Rate Limiting	express-rate-limit: 100 requests per 15-minute window per IP
Audit Trail	All create, edit, delete, login, and logout events persisted to AuditLog collection

6.2 Encryption Design

All passwords are hashed using bcrypt before storage. JWT tokens are signed using HMAC SHA-256 with a server-side secret key stored in environment variables. API communication is served over HTTPS/TLS in production environments.

6.3 Input Validation Design

express-validator middleware validates and sanitizes all incoming request fields. Validation rules enforce: email format, password minimum length, required fields, numeric field constraints, and enum value membership for role, status, and type fields.

6.4 Multi-Tenancy Design

Every MongoDB document in all collections includes a company field. Every database query is filtered by company field matching the authenticated user's company. This ensures complete data isolation between pharmaceutical companies using the shared platform.

7. Design Overlays

7.1 Security Overlay

The Security Overlay illustrates the JWT authentication workflow, RBAC middleware chain, bcrypt password hashing flow, input validation pipeline, rate limiting enforcement, and audit trail generation. This overlay is applied to the Context and Interface design views.

7.2 Observability Overlay

The Observability Overlay represents Winston logger integration, rotating daily log files, health check endpoint (/api/health), MongoDB query logging in development mode, and rate limit monitoring for blocked requests. This overlay applies to all service components.

7.3 Multi-Tenancy Overlay

The Multi-Tenancy Overlay shows how the company field propagates through all MongoDB queries, ensuring that every operation — from user listing to inventory queries to attendance records — is scoped to the authenticated user’s tenant. This overlay applies to the Information and Composition views.

8. Design Rationale

8.1 MERN Stack Architecture Decision

The MERN stack (MongoDB, Express.js, React.js, Node.js) was selected to enable full-stack JavaScript development, allowing the team to share expertise across frontend and backend. JavaScript's asynchronous I/O model in Node.js is well-suited for the concurrent API requests of a multi-user pharmaceutical management platform. This stack has a mature ecosystem for authentication, validation, and logging requirements.

8.2 MongoDB over Relational Database

MongoDB was selected over a relational database because pharmaceutical packaging operational data (inventory items with variable specifications, attendance records with computed fields, tasks with foil specifications) is well-suited to flexible document storage. The Mongoose ODM provides schema validation, enforcing data integrity at the application layer while preserving schema flexibility for future enhancements.

8.3 MVC Design Pattern

The MVC pattern was adopted to clearly separate UI rendering (React.js View), business logic processing (Express.js Controllers and Services), and data access (Mongoose Models). This separation improves testability — controllers can be tested independently of views and models — and enables parallel development across frontend and backend teams.

8.4 Service Layer for Business Logic

A Service Layer pattern was introduced for salary computation and reporting to encapsulate business logic outside of route controllers. This prevents controllers from becoming monolithic, enables unit testing of business rules independently of HTTP handling, and allows the same business logic to be called from multiple controllers without code duplication.

9. Design Languages

The Smart Pharma SDD uses the following standardized design languages:

- Unified Modeling Language (UML) — For use case diagrams, class diagrams, sequence diagrams, and state machine diagrams.
- Entity-Relationship Diagrams (ERD) — For MongoDB collection structure and relationship visualization.
- REST API Specification — For endpoint definitions, request/response schemas, and authentication requirements.
- JSON Schema Notation — For MongoDB document structures and API payload definitions.
- Mongoose Schema Definition Language — For collection schema enforcement within the ODM layer.
- Pseudocode and Flowcharts — For salary computation algorithm, QR generation logic, and attendance calculation procedures.

10. Summary

10.1 Key Design Characteristics

- Three-tier MERN stack architecture with clear separation of presentation, business logic, and data layers.
- MVC design pattern enforced across all six functional modules for consistency and maintainability.
- JWT-based stateless authentication with RBAC middleware protecting all routes.
- Multi-tenant data isolation via company field on all MongoDB collections.
- Comprehensive audit trail capturing all CRUD and authentication events.
- Modular service layer decoupling business logic from HTTP controllers.
- High cohesion and low coupling enforced across all six modules.

10.2 Outstanding Design Items

- AI Production Forecasting — Machine learning model for inventory demand prediction.
- Mobile Application — React Native companion app for workers.
- Barcode Scanner Integration — Hardware scanner support for real-time inventory validation.
- Cloud Deployment — Migration to AWS or Azure with MongoDB Atlas and auto-scaling.
- Multi-language Support — Internationalization (i18n) for Tamil and other regional languages.
- JWT Refresh Tokens — Sliding session support for improved user experience.

Annex A

(informative)

Bibliography

- [B1] IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions, Institute of Electrical and Electronics Engineers, New York, 2009.
- [B2] IEEE Std 12207-2008, Systems and software engineering — Software life cycle processes, ISO/IEC 12207:2008, Institute of Electrical and Electronics Engineers, 2008.
- [B3] IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, Institute of Electrical and Electronics Engineers, New York, 1998.
- [B4] Meta Open Source, ‘React.js Documentation,’ 2024. Available: <https://react.dev/>
- [B5] OpenJS Foundation, ‘Node.js Documentation,’ 2024. Available: <https://nodejs.org/docs/>
- [B6] OpenJS Foundation, ‘Express.js Documentation,’ 2024. Available: <https://expressjs.com/>
- [B7] MongoDB, Inc., ‘MongoDB Documentation,’ 2024. Available: <https://www.mongodb.com/docs/>
- [B8] Mongoose, ‘Mongoose Documentation,’ 2024. Available: <https://mongoosejs.com/docs/>
- [B9] Auth0, ‘JWT.io — JSON Web Token Introduction,’ 2024. Available: <https://jwt.io/introduction/>
- [B10] Provos, N., and D. Mazieres, ‘A Future-Adaptable Password Scheme,’ USENIX Annual Technical Conference 1999 — bcrypt hashing methodology.
- [B11] Bharatraj and Yokesnandha, ‘Smart Pharma System — Requirements Specification,’ CSE23CL302, SRIHER, June 2026.

Annex B

(informative)

Conforming Design Language Descriptions

This annex provides conforming design language descriptions for the two principal design languages used in this SDD beyond standard UML, per IEEE Std 1016-2009 Annex B format.

B.1 REST API Specification Language

Language Name	REST API Specification
Reference	Roy Fielding, ‘Architectural Styles and the Design of Network-based Software Architectures,’ 2000; RFC 7231 (HTTP/1.1); RFC 7159 (JSON)
Design Concerns	Interface specification: HTTP endpoints, methods, request/response schemas, status codes, authentication, rate limiting
Design Entity Types	Endpoint, Method (HTTP verb), Request Schema (JSON payload), Response Schema (JSON response), Status Code, Authorization Header
Design Attributes	URL path, HTTP method, required/optional fields, field types, constraints, authentication requirements, timeout values
Design Relationships	Endpoint → consumes → RequestSchema; Endpoint → produces → ResponseSchema; Endpoint → requires → JWTAuth

B.2 MongoDB Mongoose Schema Specification Language

Language Name	MongoDB Mongoose Schema Specification
Reference	Mongoose.js Documentation, 2024; MongoDB BSON Type System Documentation
Design Concerns	Data integrity: schema enforcement, type validation, required fields, enum constraints, multi-tenancy field, audit timestamps
Design Entity Types	Collection (schema), Field (name, type, required, enum, ref), Index, Virtual, Middleware (pre/post hooks)
Design Attributes	field name, BSON type, required boolean, default value, enum values, ref collection, unique constraint, index
Design Relationships	Collection → references → Collection (via ObjectId ref); Schema → embeds → SubDocument; Collection → indexed by → company+date

Annex C

(informative)

SDD Structure Map — Smart Pharma to IEEE 1016-2009

This annex maps the Smart Pharma SDD structure to the template structure defined in IEEE Std 1016-2009 Annex C (Figure C.1), demonstrating conformance.

IEEE 1016-2009 Template Element	Smart Pharma SDD Section
Frontispiece (date, status, issuing org, authorship, change history)	Cover Page + Change History (front matter)
Introduction — Purpose, Scope, Context, Summary	Section 1 (1.1 Scope, 1.2 Purpose, 1.3 Audience, 1.4 Conformance)
References	Section 1.5 References
Glossary	Section 2 Definitions
Body — Identified stakeholders and design concerns	Section 3.2 Design Stakeholders and Their Concerns
Body — Design viewpoint 1 (Context)	Section 5.2 Context Viewpoint
Body — Design viewpoint 2 (Composition)	Section 5.3 Composition Viewpoint
Body — Design viewpoint 3 (Logical)	Section 5.4 Logical Viewpoint
Body — Design viewpoint 4 (Dependency)	Section 5.5 Dependency Viewpoint
Body — Design viewpoint 5 (Information)	Section 5.6 Information Viewpoint
Body — Design viewpoint 6 (Interface)	Section 5.7 Interface Viewpoint
Body — Design viewpoint 7 (Interaction)	Section 5.8 Interaction Viewpoint
Body — Design viewpoint 8 (State Dynamics)	Section 5.9 State Dynamics Viewpoint
Body — Design viewpoint 9 (Algorithm)	Section 5.10 Algorithm Viewpoint
Body — Design viewpoint 10 (Resource)	Section 5.11 Resource Viewpoint
Design overlays	Section 7 Design Overlays
Design rationale	Section 8 Design Rationale
Annex A — Bibliography	Annex A Bibliography
Annex B — Conforming design language description	Annex B Design Language Descriptions
Annex C — Templates for an SDD	Annex C (this annex) — SDD Structure Map